Home

虽然 IT 领域中各种新框架、新技术层出不穷，但它们的根基还是那些计算机基础知识。没有扎实的计算机基础知识，应用层的技术学习就像空中楼阁，只能看到表象却无法理解本质，学习的时候自然是一头雾水。

正所谓“磨刀不误砍柴工”，优秀工程师的共同特质就是扎实深厚的计算机基础知识，相信工作过一段时间的人都能意识到它的重要性。

尽管知道扎实的基本功对开发效率、技术成长都很重要，但是我们学习时遇到的问题也很多，通常困惑于以下三类问题：

- 基础是什么：对基础知识到底包括哪些，需要学到什么程度心里没底。
- 基础怎么学：底层技术那么多，内容繁多不知从何下手，缺乏清晰系统的学习路径。
- 理论如何和应用相结合：脑子说会了，手说“你没有”，学习技术的时候看了很多资料仍然无法融会贯通，更别说应用到工作之中。

为此，我们邀请到彭东老师，他既做过前端、后端的工作，也做过内核的开发。在这门课里，老师将结合自己十多年的理论和实战经验，帮你从源头开始一步步夯实基本功：从硬件到软件、从指令语言到内存，从原理到应用，带你系统了解计算机的基础知识。

学完这门课程，你就掌握了 IT 领域 80% 相对稳定不变的那部分知识，对于学习任何一门应用技术也是很好的前置准备。

课程设计

串联计算机从底层到应用的关键知识点，总计四个核心、八大关卡。

核心一：历史

学计算机基础，需要先学习它的历史，学习计算机是怎么一步步发展到今天这个样子的，以今天的状况推导未来的发展方向。我们会重点讨论编程架构是怎么创造出来的、CPU 从何而来、CISC 和 RISC 又各有什么优缺点。

核心二：硬件基础知识

了解硬件的工作机制对工程师实现优秀的应用软件非常重要。这里不但会详细讲解芯片的工作机制与原理，还会带你手把手实现一个五级流水线的迷你 RISCV 处理器（基于Verilog，配套代码量2882行）。

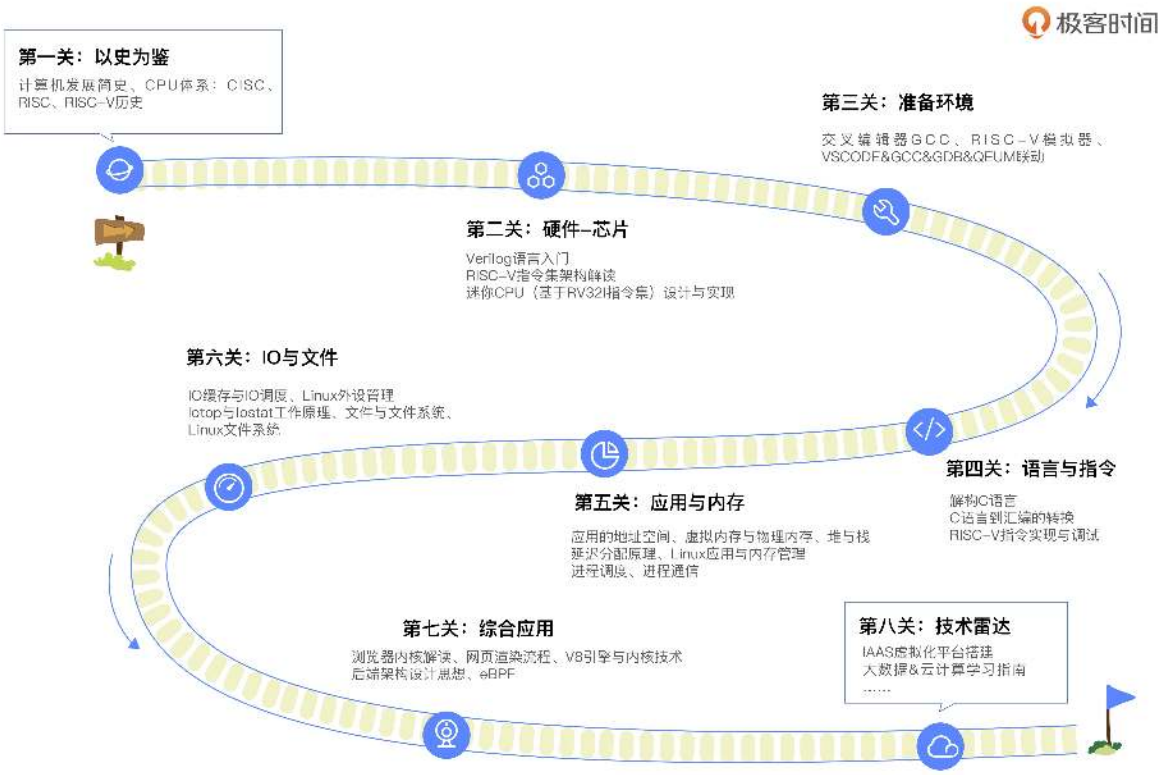
核心三：软件基础知识

想要开发高性能的应用程序，语言与指令、应用与内存、IO 与文件方面的知识都是必修内容。这里我们会通过第三关到第六关四个章节带你掌握这些计算机软件基础中最核心、最重要的知识。

高级语言和低级语言之间如何互相转化？应用堆和栈内存有什么不同？多个进程之间如何通信？Linux如何管理外设，又如何存放文件……这些重难点问题，一个都不会漏掉。

核心四：应用层基础知识

带你从底层角度审视前端技术跟后端架构，并探讨云计算、大数据与智能制造。这些热门领域都是对基础技术的综合应用，有助于你开阔视野，提升自己的知识迁移能力，让你真正学有所用。



开篇词｜练好基本功，优秀工程师成长第一步

你好，我是彭东，网名LMOS。很高兴在极客时间和你相遇，一起开启计算机基础的修炼之旅。

先来介绍一下我自己。我是 Intel 傲腾项目开发者之一，曾经为 Intel 做过内核层面的开发工作，也对 Linux、BSD、SunOS 等开源操作系统，还有 Windows 的 NT 内核很熟悉。

这十几年来，我一直专注于操作系统内核研发。先后开发了LMOS（基于x86_64的多进程支持SMP的操作系统）和LMOSEM（基于ARM32，支持软实时的嵌入式操作系统），还写过《深度探索嵌入式操作系统》一书。去年5月份，我在极客时间上更新了 《操作系统实战45讲》 这个专栏，和你分享了我多年来开发操作系统的方法和经验。

通过课程的互动交流，我发现很多同学因为基础知识并不扎实，所以学操作系统的时候非常吃力。而计算机的基础知识，不但对于深入理解操作系统有帮助，对我们工程师的技术提升也是一门长期收益的必修课。

打牢计算机基础有什么用?

就拿我的亲身经历来说，我既做过前端、后端的工作，也做过内核的开发。出现Bug和故障的时候，我总能快速理清排查思路，选用合适的工具、技术来分析问题，高效Debug；一个项目摆在我面前，迅速分析出项目的痛点、难点，整理出实现功能需要哪些技术框架也是驾轻就熟。

很多同事跟朋友对这样的能力心向往之，好奇我有什么“秘诀”。其实，能来回穿梭于底层与高层之间，不至于手忙脚乱，我最大的依仗就是深厚的计算机基础。

无论你是计算机初学者，还是已经工作了几年的老同学，对于“打牢基础很重要”、“基础不牢、地动山摇”这样的话，估计耳朵都要听得磨出茧子了。但到底计算机基础威力有多大呢?

举个例子，就像你编写你人生的第一个程序——Hello World。这个程序非常简单，同时也非常复杂，简单到你只要明白调用函数“printf(“Hello World\n”);”，就能在屏幕上打印出Hello World的字符；难的是这个程序的背后细节，尽管这个程序不过数行代码，却需要芯片、编程语言、进程、内存、IO等多种基础设施的配合，才能完成看似简单的功能。

当然在写Hello World程序这个起步阶段，我们只要知道printf函数如何使用就行了，这是因为这程序简单到只是输出Hello World就结束了，不会给系统或者其它软件带来副作用。

但若是我们要开发大规模应用系统，如电商服务系统，问题就会变得复杂。比如：

1.这个服务应用要用什么语言来编写?

2.是采用单体进程，还是用多个进程来协同工作?

3.如何管理长期使用的内存空间？如何避免系统IO抖动?

4.如何处理网络带来的各种问题，比如通信拥堵、拒绝请求，甚至掉线?

这些问题，显然不是我们知道这些方面的几个接口函数就能解决的。发现没有？你可以用很短的时间跑起来一个Hello World，但想保障一个电商系统运转如常，感觉难度上是天壤之别。工程复杂度带来的差异，让我们不得不继续钻研，试着“理解”计算机。

我再说一个MySQL的例子：在往生产数据库中导入部分数据时，会造成客户端的访问超时。你可能怀疑这是MySQL自身问题，也可能怀疑是服务器系统的问题。其实两者都不是，此时即使你对MySQL的各种操作都了然于胸，还是对解决这类问题一头雾水。

如果你没能掌握文件系统、Cache、IO等基础的话，就很难想到用iotop、iostat等工具去查看IO操作，也就无从发现MySQL在导入数据时还会产生大量的日志，而这些日志也需要存盘引发大量IO操作，导致IO带宽爆满，造成访问超时。更不用说想到可以用MySQL的innodb_flush_log_at_trx_commit来控制MySQL的log行为了。

再比方说，如果你不知道操作系统与CPU、RAM等硬件的交互原理，就很难理解JVM为啥要抽象出堆、虚拟机栈和本地方法栈、程序计数器、方法区之类的概念来屏蔽硬件差异，更别说理解JVM、JUC中的内存管理、多线程安全的核心设计思想了。你看，写不出高并发、安全可靠程序的瓶颈，深究起来欠缺的竟然是底层基础知识。

除了复杂的软件工程问题，日新月异的前沿技术也离不开计算机基础的软硬件知识。

系统设计领域，只有研究过对CPU提供的SIMD指令集，才会联想到可以像ClickHouse一样基于向量化执行来提升计算速度；在云原生方面，只有熟知文件系统的系统调用和运作原理，才能设计出一款优质的分布式文件系统，或者设计出基于UnionFS的Docker 镜像机制，让容器真正发挥优势；AI领域同样如此，只有透彻理解了语言与指令、内存与应用，才有可能通过基础的软硬件技术配合优化存储层次，最终调优加速AI框架……

总之， **想要成为优秀工程师，就需要你深入芯片、内存、语言、应用、IO与文件等这些基础组件学习研究**，甚至还要钻研语言指令的运转，搞懂芯片尤其是CPU的机制原理。这些基础，不仅仅是对计算机本身很重要，对从事计算机的任何细分行业的每个人都很重要。

计算机基础要怎么学?

也许你跟我一样，不是计算机专业科班出身，所以起步时更加步履维艰。通常被后面这几类问题困扰：不确定学什么，不知道怎么学，硬记了概念不明白技术原理，更别说学以致用了。

这些问题让我们面对内容繁多的计算机知识时，不知如何下手，于是开始自我怀疑，总想打退堂鼓。从只会用C写个Hello World，到可以用C语言自研操作系统内核，我同样经历了漫长的修炼之

旅。我也遇到过各种各样的问题，通过不断地学习和实践，才解决了诸多疑难杂症。

我希望把自己积累的大量计算机学习基础方法经验，通过这门课分享给你，帮你把计算机从底层到应用的关键知识点串联起来。除了学习原理概念、理顺知识点，动手实践的环节也不可或缺，配套的执行和调试代码，我之后都会放在 Gitee 上方便你随堂练习。

这个专栏我是这样安排的：

历史

一个东西，从何而来，何至于此，这就是历史。学计算机基础，我们需要先学习它的历史，学习计算机是怎么一步步发展到今天这个样子的，再根据今天的状况推导出未来的发展方向。

我并不会长篇累牍地给你讲什么编年史，而是重点带你了解可编程架构是怎么创造出来的、CPU从何而来、CISC和RISC又各有什么优缺点。知道了这些，你就能理解为什么现在国家要提倡发展芯片产业，RISC-V为何会大行其道。

芯片

万丈高楼从地起，欲盖高楼先打地基。芯片是万世之基，这是所有软件基础的开始，执行软件程序的指令，运算并处理各种数据都离不开它。

因此，了解芯片的工作机制对写出优秀的应用软件非常重要。为了简单起见，我选择了最火热的RISCV芯片。这个模块里，我们将一起设计一个迷你RISCV处理器。哪怕未来你不从事芯片设计工作，了解芯片的工作机制，也对写出优秀的应用软件非常重要。

环境

学习讲究“眼到，手到，心到”，很多知识如果想牢牢掌握，就离不开动手实践。

而搭建好编译环境和执行环境就是实践的前提，方便后面的学习里我们去调试程序，验证理论。环境篇我们最终会跑出RISC-V平台的Hello World程序，作为这一关的阶段性成果。

语言

一个合格的程序员必须要掌握多种编程语言，这是开发应用软件的基础，所以我选择了最常用的C语言，以它为例让你理解高级语言是如何转换成低级的RISCV汇编语言的。

我不光会带你学习C语言各种类型的形成、语句与函数的关系，还会给你搭建一座理解C和汇编对应关系的桥梁。汇编语言方面，我会以RISC-V为例，介绍其算术指令、跳转指令、原子指令和访存指令，并带你学会调试这些指令，加深你对指令的理解。

应用

具备了编程语言的知识基础，我们就可以开发应用了。应用往往与内存分不开，我们一起来了解应用的舞台——内存地址空间，接着会引入物理内存、虚拟内存。理解了内存，理解进程也会手到擒来。

虚拟内存跟物理内存如何映射和转换？应用堆和栈内存有什么不同？应用内存是如何动态分配的？为什么操作系统中能并行运行多个不同或者相同的应用？多个应用之间如何通信？这些重难点问题，我们一个都不会漏掉。

IO

跟软件应用直接关联的，除了芯片和内存之外，就是IO即输入输出系统了。无论是交互式应用、还是数据密集型应用，都不得不接收各种数据的输入，然后执行相应计算和处理之后产生输出。

有的应用性能不佳，实时性不强，更有甚者丢失数据，面对这些令人头疼的问题，不懂IO就无法处理。我们想要开发高性能的应用程序，就不得不学习IO相关的基础知识了。因此，我们会重点学习IO的操作方式、IO调度、IO缓存Cache，以及Linux操作系统是如何管理IO设备的。我还会引入iotop和iostate工具，带你掌握怎么用它们来攻克应用的IO性能瓶颈。

文件

很少有应用不需要储存读写文件的，特别是各种网络应用和数据库应用，一个合格的开发者必须对文件了如指掌。

想要提升应用读写数据性能，做好数据加密（特别是优化网络数据库应用），深入了解文件和文件系统都是相当关键的。理清文件的基础知识点之后，我们还会研究一个Linux文件系统实例的内部细节，检验之前所学。

综合应用

经历了前面这些关卡，在综合应用篇里，我会带你了解如何从底层角度审视前端技术跟后端架构。优秀工程师通常具备超强的知识迁移能力，能够透过各种多变的技术表象，快速抓住技术的本质。这将是你未来拓展学习更多应用层技术，顺利解决日常业务里前后端性能问题的良好开端。

技术雷达

最后，我还设置了技术雷达的加餐内容，和你聊聊云计算、大数据跟智能制造。这些热门领域其实都是对基础技术的综合应用，有助于你开阔视野，给工作选择增加更多可能性。

这个加餐，我安排在正文结束之后的一个月和你见面（每周更新一节课，共五节课），这一个月是留给你吸收消化前面所学内容的时间。

总之，在你学习更多应用层技术以前，通过这门课补充前置知识很有必要。这既是所有有志于成为高手的工程师绕不开的必修内容，同样也是我多年职业生涯里，通过技术修炼沉淀而来的“学习笔记”。

在我看来， **一个人的自我学习能力和态度决定着技术成就，不然只会陷入CRUD Boy或者API Caller的圈子里，终日忙忙碌碌却依旧原地踏步。IT人就是要时刻保持学习，如果要给这个保持学习的习惯加个期限，那就是“终身”。**

期待你加入其中，我们一起进步，共同成长！ 戳此加入课程交流群

可以自己立个 Flag，哪怕只是在留言区打卡你的学习天数、今日的问题或者收获，我相信都是会有效果的。3 个月后，我们再来一起验收。

404 - Not found

02｜RISC特性与发展：RISC V凭什么成为“半导体行业的Linux”？

你好，我是LMOS。

上节课，我带你见证了两种计算机指令集的设计结构——CISC与RISC。而今天我们的“主角”就是RISC中的一个代表性特例，它就是RISC-V。

作为未来芯片指令集的主流，RISC-V是今后芯片设计的最佳方案，甚至可以说它就是硬件行业里的Linux。

为什么这么说呢？这节课，我会从RISC-V发展历史、原理与技术特性几个方面入手，带你弄明白为什么RISC-V在半导体行业中发展得如此迅猛。

RISC-V从何而来

让我们“穿越时空”，把时间线拉回到2010年。在加州伯克利分校的校园中，Krste Asanovic教授正为了学生们学习计算机架构而发愁。由于现存芯片指令集冗余且专利许可费用昂贵，还有很多IP法律问题，没有一款合适的CPU用于学习。

于是他想要带领一个研究团队，来设计一款用于学生学习的CPU。研究团队在选择架构的时候，对比了传统已经存在的ARM、MIPS、SPARC以及x86架构等，发现这些架构指令集要么极其复杂，要么极其昂贵。所以，他们的研究团队最终决定设计一套全新的指令集。

这个研究团队最开始只有4个人，却在三个月之内完成了指令集原型开发，其能力可见一斑。后来，这个项目被计算机体系结构方面的泰斗 David Patterson 发现，并且得到了他的大力支持。

早在1981年，伯克利分校已经设计出了第一代RISC指令集，之后又陆续设计了四代RISC指令集的架构。有了这些设计经验，在设计RISC-V指令集架构时，研究团队就变得驾轻就熟。

用RISC-V来命名该指令集架构，有两层意思：RISC-V中的“V”，一方面代表第5代RISC；另一方面，“V”取“ Variation”之意，代表变化。

虽然RISC-V在2010年才开始研发，但在第二年（2011年）就进行了首次流片，流片就是按照芯片设计生产出可以工作的芯片成品。从这以后，基于RISC-V的CPU设计，或者在SOC中集成RISC-V架构，各种软件工具链的开发和各种基于RISC-V架构的操作系统移植，在不断涌现。这给CPU整个行业带来了不小轰动，当然，这也离不开泰斗 David Patterson 的号召作用。

一个产品的成功，技术当然是非常重要的原因，但也需要非常给力的运营。在2015年，由最初的25个成员一起成立了非盈利性组织RISC-V基金会（RISC-V Foundation）。

之后，有多达300个单位加入RISC-V基金会，其中包括阿里、谷歌、华为、英伟达、高通、麻省理工学院、普林斯顿大学、印度理工学院、中科院计算所、联发科等。这些学术机构、芯片开发公司、设计服务与系统厂商的加入，为RISC-V的发展建立了良好的生态环境。

RISC-V是什么

通过上节课，我们知道了RISC就是指精简指令集计算机体系结构。而前面也说了，RISC-V是伯克利分校设计出的第五代RISC指令集架构。但既然迅速走红的是RISC-V，相比其他的RISC，它必然有过人之处，这样才能立足于市场，不然很可能只沦为学生们的学习工具。

如果只是对RISC-V下一个定义，会相当简单： **RISC-V是一套开放许可证书、免费的、由基金会维护的、一个整数运算指令集外加多个扩展指令集的CPU架构规范。**

任何硬件开发商或者相关组织都可以免费使用这套规范，构建CPU芯片产品。如果我们的目的只是想对RISC-V有个概念了解，前面这些信息就足够了。不过想知道RISC-V为什么流行这个秘密，显然我们还需要更详细的信息，才能深入了解。

指令集命名方式

现在假定我们是一家芯片公司的芯片工程师，公司有了新的业务，对CPU提出了更高的要求，让我们基于RISCV指令集架构实现一款全新的CPU。根据我们公司的业务场景，我们只需要选择RISC-V架构中的一部分指令，CPU的位宽也有特定的要求。

因此，现在需要一些命名方式，来对我们选择的RISCV架构进行命名。这样，用户在阅读该CPU文档时，马上就能了解这款CPU是多少位的、有哪些指令集，它们具体有什么功能。

其实这个命名方式在RISC-V规范中有相关定义说明，以RV为前缀，然后是位宽，最后是代表指令集的字母集合，具体形式如下：

```
RV[###][abc……xyz]
```

我用表格为你说明一下这个格式，如下所示：图片

举个例子，比如：RV64IMAC，就表示64位的RISC-V，支持整数指令、乘除法指令、原子指令和压缩指令。

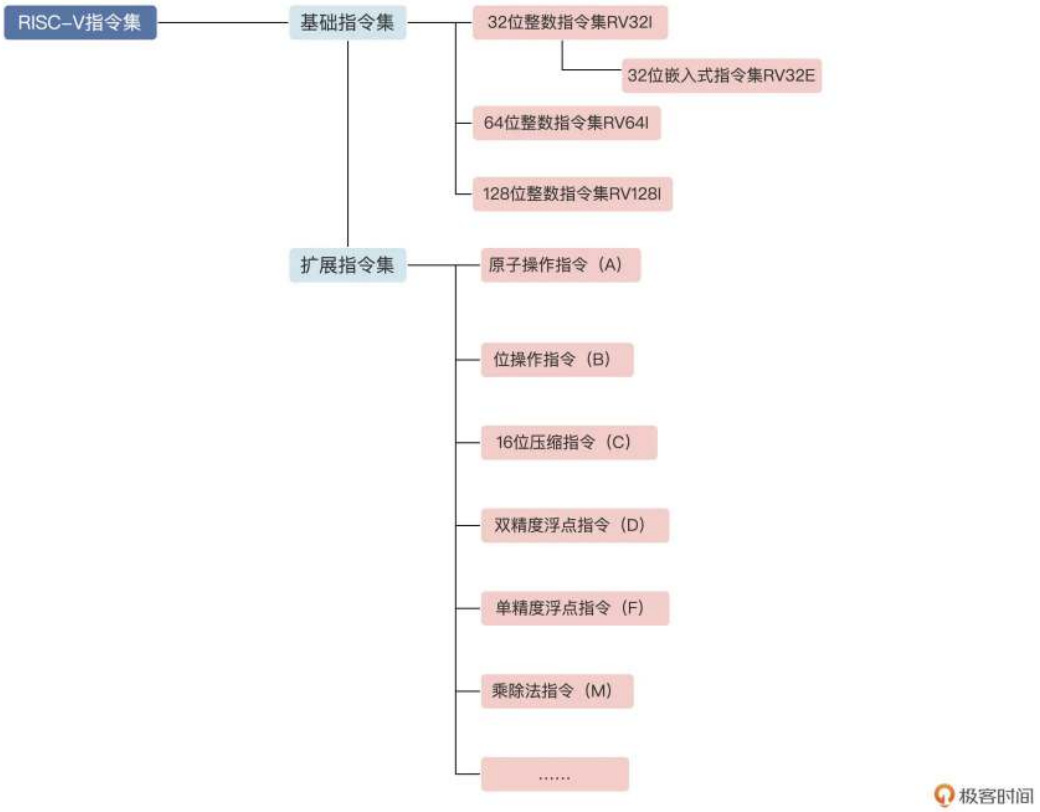
指令集模块

接着我们一起来看看指令集模块。指令集是一款CPU架构的主要组成部分，是CPU和上层软件交互的核心，也是CPU主要功能的体现。

但RISC-V规范只定义了CPU需要包含基础整形操作指令，如整型的储存、加载、加减、逻辑、移位、分支等。其它的指令称为可选指令或者用户扩展指令，比如乘、除、取模、单精度浮点、双精度浮点、压缩、原子指令等，这些都是扩展指令。扩展指令需要芯片工程师结合功能需求自定义。

所以RISC-V采用的是模块化的指令集，易于扩展、组装。它适应于不同的应用场景，可以降低CPU的实现成本。

我给你列了一张图，图里展示的是RISC-V 指令集的各个组成部分。



RISC-V的基本指令集和扩展指令集你有个大致印象就可以，更详细的技术讲解，后面第五节课我再展开。下面我们去看看RISC-V的寄存器。

RISC-V寄存器

指令的操作数是来源于寄存器，精简指令集架构的CPU，都会提供大量的寄存器，RISC-V当然也不例外。RISC-V 的规范定义了32个通用寄存器以及一个 PC寄存器，这对于RV32I、RV64I、RV128I指令集都是一样的，只是寄存器位宽不一样。

如果实现支持 F/D 扩展指令集的CPU，则需要额外支持 32个浮点寄存器。而如果实现只支持RV32E指令集的嵌入式CPU，则可以将32个通用寄存器缩减为16个通用寄存器。

寄存器	ABI名称	说明
x0	zero	0值寄存器，硬编码为0，写入数据忽略，读取永远为0
x1	ra	用于返回地址
x2	sp	用于栈指针
x3	gp	用于通用指针
x4	tp	用于线程指针
x5	t0	用于存放临时数据或者备用链接寄存器
x6~x7	t1~t2	用于存放临时数据寄存器
x8	s0/fp	需要保存的寄存器或者帧指针寄存器
x9	s1	需要保存的寄存器
x10~x11	a0~a1	函数传递参数寄存器或者函数返回值寄存器
x12~x17	a2~a7	函数传递参数寄存器
x18~x27	s2~s11	需要保存的寄存器
x28~x31	t3~t6	用于存放临时数据寄存器

极客时间

为了帮助你聚焦要点，不常用的32个浮点寄存器并没有列在这张表里。表中的ABI名称，即应用程序二进制接口，你可以理解为寄存器别名，高级语言在生成汇编语言的时候会用到它们。

比如C语言处理函数调用时，用哪些寄存器传递参数、返回值，调用者应该保护哪些寄存器，用什么寄存器管理栈帧等等。

定义好ABI标准，我们就能在语言间互相调用函数了。比如C语言函数调用汇编语言函数，这里我先卖个关子，后面语言与指令的篇章再给你详细展开。

RISC-V特权级

研究完了RISC-V寄存器，我们再来看看RISC-V的特权级。不同的特权级能访问的系统资源不同，高特权级能访问低特权级的资源，反之则不行。RISC-V 的规范文档定义了四个特权级别（privilege level），特权等级由高到低排列，如下表所示。

名称	级别	缩写	编码
用户、应用程序特权级	0	U	00
管理员特权级	1	S	01
虚拟机监视特权级	2	H	10
机器特权级	3	M	11

极客时间

一个RISC-V硬件线程（hart），相当于一个CPU内的独立的可执行核心，在任一时刻，只能运行在某一个特权级上，这个特权级由CSR（控制和状态寄存器）指定和配置。

具体分级如下：

1.机器特权级（M）：RISC-V中hart可以执行的最高权限模式。在M模式下运行的hart，对内存、I/O和一些必要的底层功能（启动和系统配置）有着完全的控制权。因此，它是唯一一个所有标准RISC-V CPU都必须实现的权限级。实际上，普通的RISC-V微控制器仅支持机器特权级。

2.虚拟机监视特权级（H）：为了支持虚拟机监视器而定义的特权级。

3.管理员特权级（S）：主要用于支持现代操作系统，如Linux、FreeBSD和Windows。

4.用户应用特权级（U）：用于运行应用程序，同样也适用于嵌入式系统。

好了，关于RISC-V的特权级，你了解这些，在现阶段已经足够了。需要把握的重点是，特权级起到了怎样的作用。

有了特权级的存在，就给指令加上了权力，从而去控制用指令编写的程序。应用程序只能干应用程序该干的事情，不能越权操作。操作系统则拥有更高的权力，能对系统的资源进行管理。

RISC-V因何流行

RISC-V指令集架构在2010年才开发出来，到今天不过10多年的时间。这个时间从CPU行业的发展看，是非常短的，也可以说是非常年轻的。相比x86的40多岁的年纪，还有ARM、MIPS、SPARC的30多的年纪，RISC-V简直是个孩子。

要知道ARM、MIPS、SPARC都是RISC系的，MIPS和SPARC甚至已经进入了死亡阶段。按道理讲，RISC-V不应该在这么短的时间内流行起来，成为芯片行业一颗耀眼的新星。

那么RISC-V流行起来，肯定有其优势： **一是RISC-V完全开放，二是RISC-V指令简单，三是RISC-V实行模块化设计，易于扩展。**

我们先来看看为什么说RISC-V是开放的。之前硬件和软件一样，都是小心地保护自己的“源代码”，因为那是自己的命脉。

直到后来，软件界出现了开源的Linux，一经开源就迅速席卷了全球。在今天的互联网、云计算、手机等领域Linux已经无处不在。但是硬件依然保护着自己的“源代码”，Intel和AMD还是以售卖x86芯片为主，而ARM直接售卖ARM CPU的“源代码”，连生产芯片的步骤都省了。

这种模式下，无论厂商还是个人，要获得CPU都要付出昂贵的代价。这时RISC-V应运而生，它完全毫无保留地开放了CPU设计标准，任何人都可以使用该标准，自由地设计生产CPU，不需要支付任何费用，也没有任何法律问题。这相当于硬件界的“Linux”，推动了开放硬件的运动和发展。

然后，我们来看看为什么说RISC-V很简单？RISC-V提供了一个非常强大且开放的精简指令集架构，只有32个通用寄存器、40多条常用指令、4个特权级。如果需要其它功能，则要进行指令集的扩展，单核心的规范文档才不到300页，一个人在一周之内就能搞清楚。

相比ARM、x86动不动就有8000多页的规范文档，这实在是太简单了。其实，简单也意味着可靠和高效，同时可以让学生或者硬件开发者迅速入手，降低学习和开发成本。

最后，我们来说说RISC-V的模块化设计。RISC-V虽然简单，但这并不意味着功能的缺失。通过模块化的设计，就能实现对各种功能组件的剪裁和扩展。

事实上，现代IT架构已经发生了巨大的改变。举几个我们身边的例子吧。你正在使用的网卡，上面越来越多的网络处理任务和功能，都从主处理器上移到了网卡中，由网卡芯片自己来处理了。

数据处理器 (DPU) 也体现了这一点。由于通用处理器对大规模数据处理能力的限制，所以我们需要专用的数据处理器。而人工智能领域，现在也已经开始通过GPU运行相关算法。

这些例子都在告诉我们，专用处理器芯片的需求在大量激增，而这正是RISC-V的用武之地。RISC-V的标准开放，指令功能模块可以自由组合，所以用RISC-V就能定制一款满足特殊用途的处理器。芯片工程师会自由组合RISC-V现有的指令功能模块，按需对齐进行修改优化，或者实现一个新的指令功能模块，就像你根据需要修改和使用Linux内核一样。

正是因为RISC-V开放、简单和模块化这三大特点，硬件工程师和软件工程师才能站在巨人的肩膀上开发，自由地调用和组装功能模块，快速去实现特定业务场景下的芯片需求。因此，才有了RISC-V引爆芯片行业，迅速火热起来的现象，这是推动开放硬件的革命性壮举。

重点回顾

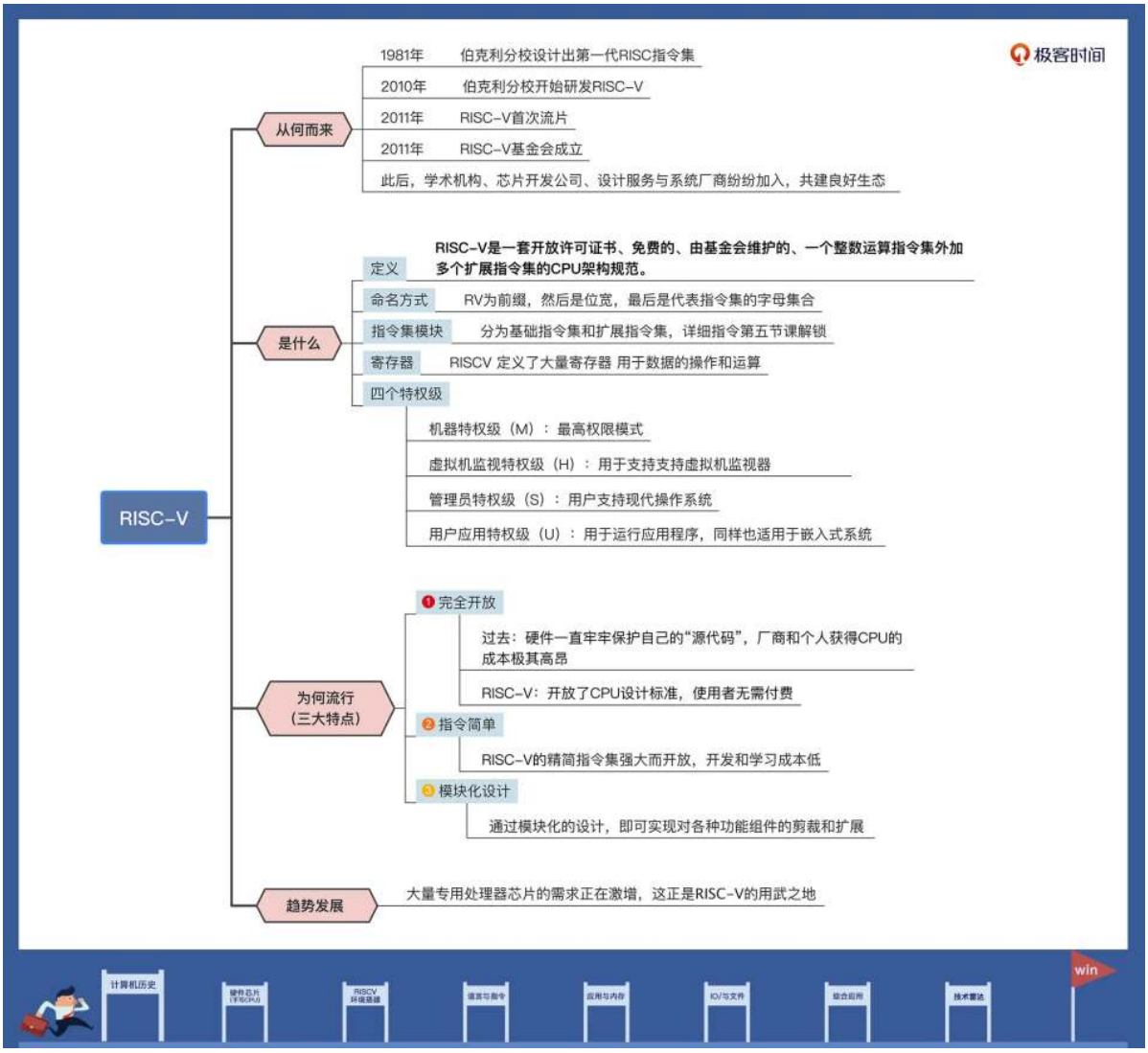
今天的课程又到了尾声，我们还是来看一下，在这节课中，我们都学习了什么。

首先，我们了解了RISC-V从何而来，明白了RISC-V发源于加州伯克利分校，是该校第五代RISC指令集，即第五代精简指令集。起初是为了学生有一套用来学习研究的指令集。后来因为有技术泰斗David Patterson的加入，又成立RISC-V基金会，RISC-V慢慢流行了起来。

之后我们研究了RISC-V是什么，我带你了解了RISC-V指令集的命名方式、组成模块、寄存器与特权级。这些部分共同组成了RISC-V指令集架构规范。任何硬件厂商都可以按照这个规范，实现自己的RISC-V处理器。

最后，我们讨论了RISC-V因何流行。RISC-V是开放的，没有任何法律和许可证问题，又极其简单，指令集是模块化的，易于剪裁和扩展。这种开放、简单、易于扩展的特点，使得硬件工程师非常容易上手和定制满足特定功能需要的处理器，这直接推动了开放硬件的革命。

课程里的重点内容，我整理成了导图，供你参考。



思考题

为什么RISC-V要定义特权级?

欢迎你在留言区跟我交流互动，如果觉得内容还不错，也推荐你把这节课分享给更多朋友。下节课我们就进入手写miniCPU的部分，敬请期待！

先睹为快：迷你CPU项目效果演示

你好，我是LMOS。

这个模块里，我们将一起设计一个迷你 RISCV 处理器。我准备了演示视频，让你先睹为快，看看我们最后做出的迷你CPU长什么样子。

看了视频，你有什么感受呢？先别急着动手敲代码，我们接下来会通过九节课的时间，从Verilog语言讲起，然后再进行迷你CPU的设计和实现。

希望通过这一阶段的学习，为你铺垫一些硬件的基础知识，也给你推开RISC-V的一扇窗。一起加油，希望在后面的留言区看到你的身影。

03｜硬件语言筑基（一）：从硬件语言开启手写CPU之旅

你好，我是LMOS。

我们都知道，自己国家的芯片行业被美国“吊打”这件事了吧？尤其是像高端CPU这样的芯片。看到相关的报道，真有一种恨铁不成钢的感觉。你是否也有过想亲自动手设计一个CPU的冲动呢？

万丈高楼从地起，欲盖高楼先打地基，芯片是万世之基，这是所有软件基础的开始，这个模块我会带你一起设计一个迷你RISC-V处理器（为了简单起见，我选择了最火热的RISCV芯片）。哪怕未来你不从事芯片设计工作，了解芯片的工作机制，也对写出优秀的应用软件非常重要。

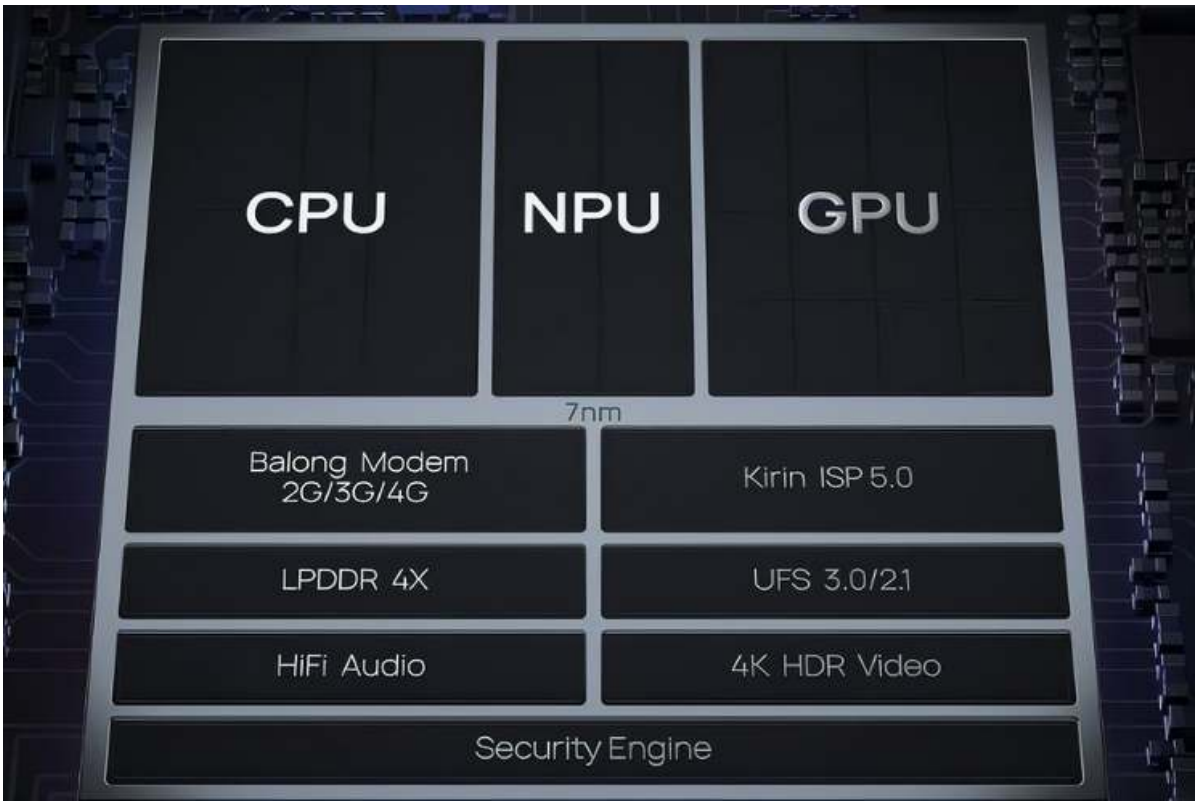
这个处理器大致是什么样子呢？我们将使用Verilog硬件描述语言，基于RV32I指令集，设计一个32位五级流水线的处理器核。该处理器核包括指令提取单元、指令译码单元、整型执行单元、访问存储器和写回结果等单元模块，支持运行大多数RV32I的基础指令。最后，我们还会编写一些简单汇编代码，放在设计出来的处理器上运行。

我会通过两节课的篇幅，带你快速入门Verilog，为后续设计迷你CPU做好准备。这节课我们先来学习硬件描述语言基础，芯片内部的数字电路设计正是由硬件语言完成的。

一个芯片的内部电路是怎么样的？

作为开发，你日常最常用的编程语言是什么？也许是C语言、Java、Go、PHP.....这些高级编译语言吧。而硬件设计领域里，也有专门的硬件描述语言。为什么会出现专门的硬件描述语言呢？这还要先从芯片的内部结构说起。

一般情况下，你所接触到的处理器芯片，已经不是传统意义上的CPU了，比如在业界很有名的国产手机芯片华为麒麟990芯片。我把麒麟990的功能模块图贴在了后面，对照图片会更直观。这样一款芯片，包含了CPU核、高速缓存、NPU、GPU、DDR、PMU等模块。



而在芯片设计时，根据不同模块的功能特点，通常把它们分为数字电路模块和模拟电路模块。

模拟电路还是像早期的半导体电路那样，处理的是连续变化的模拟信号，所以只能用传统的电路设计方法。而数字电路处理的是已经量化的数字信号，往往用来实现特定的逻辑功能，更容易被抽象化，所以就产生了专门用于设计数字电路的硬件描述语言。

硬件描述语言从发明到现在，已经有20多年历史。硬件描述语言可以让你更直观地去理解数字电路的逻辑关系，从而更方便地去设计数字电路。

现在业界的 IEEE标准主要有VHDL和Verilog HDL 这两种硬件描述语言。在高层次数字系统设计领域，大部分公司都采用 Verilog HDL完成设计，我们后面的实现也会用到Verilog。

千里之行，始于足下。在Verilog学习之前，我们需要先完成思路转换，也就是帮你解决这个问题：写软件代码和写硬件代码的最大区别是什么？搞明白了这个问题，你才能更好地领会Verilog语言的设计思想。

Verilog代码和C语言、Java等这些计算机编程语言有本质的不同， **在可综合（这里的“可综合”和代码“编译”的意思差不多）的Verilog代码里，基本所有写出来的东西都对应着实际的电路。**

所以，我们用Verilog的时候，必须理解每条语句实质上对应着什么电路，并且要从 **电路的角度** 来思考它为何要这样设计。而高级编程语言通常只要功能实现就行。

我再举几个例子来说明：声明变量的时候，如果指定是一个reg，那么这个变量就有寄存数值的功能，可以综合出来一个实际的寄存器；如果指定是一段wire，那么它就只能传递数据，只是表示一条线。在Verilog里写一个判断语句，可能就对应了一个MUX（数据选择器），写一个for可能就是把一段电路重复好几遍。

最能体现电路设计思想的就是always块了，它可以指定某一个信号在某个值或某个跳变的时候，执行块里的代码。通过使用Verilog语言，我们就能完成芯片的数字电路设计工作了。没错，芯片前端设计工程师写Verilog代码的目的，就是 **把一份电路用代码的形式表示出来，然后由计算机把代码转换为所对应的逻辑电路**。

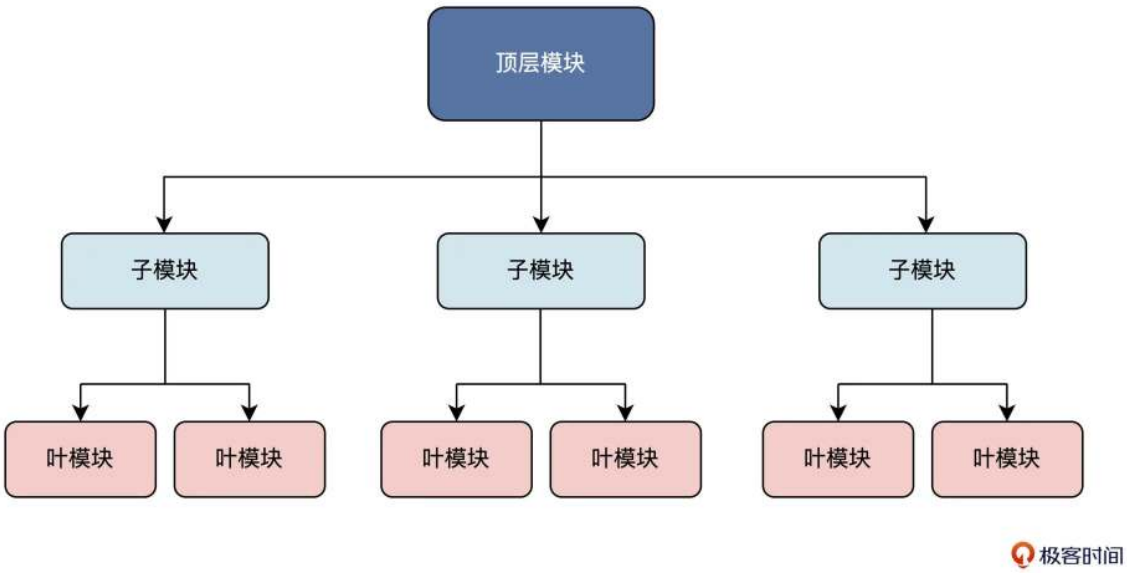
芯片如何设计?

说到这里你可能还有疑惑，听起来芯片设计也没那么复杂啊？其实这事儿说起来简单，但实践起来却相当复杂。接下来，我就说说，一个工业级的芯片在设计阶段大致会怎么规划。

在开始一个大的芯片设计时，往往需要先从整个芯片系统做好规划，在写具体的Verilog代码之前，把系统划分成几个大的基本的功能模块。之后，每个功能模块再按一定的规则，划分出下一个层次的基本单元。

这和Verilog语言的module模块化设计思想是一致的，上一层模块对下一层子模块进行例化，就像其他编程语言的函数调用一样。根据包含的子功能模块一直例化下去，最终就能形成hierarchy结构。

这种自顶向下的设计方法，可以用后面的树状结构图来表示：



从上图我们也可以看出，Verilog都是基于模块进行编写的，一个模块实现一个基本功能，大部分的Verilog逻辑语句都放在模块内部。

从一段代码入门Verilog

说完语言思路和硬件的模块化设计，接下来，我带你学习一下Verilog的基本模块和逻辑语句是怎么写的。

很多Verilog初学者刚开始都是从一些基础知识慢慢去看，比如基本语法、数据类型、赋值语句、条件语句……总想着把Verilog的全部基础知识看完了，再开始动手写代码。

但是你有没有想过，这些详细的基础知识，一两天自然是看不完的。而当你坚持了一段时间把它看完，以为可以上手写代码的时候，又会发现前面的基本语句全都忘了。这样的学习方法并不可取，效果也不好，所以我换个方法带你入门。我们先不去罗列各种详细的基础知识，而是从学习一段代码开始。

我会以一个4位十进制计数器模块为例，让你对Verilog模块代码有更直观的认识，然后根据这段代码模块，给你讲讲Verilog语言基础。这里先把完整代码列出来，后面再详细拆解。

```
module counter(
  //端口定义
  input                   reset_n,  //复位端，低有效
  input                   clk,        //输入时钟
  output [3:0]              cnt,      //计数输出
  output                  cout     //溢出位
);

reg [3:0]               cnt_r ;      //计数器寄存器

always@(posedge clk or negedge reset_n) begin
  if(!reset_n) begin                        //复位时，计时归0
      cnt_r        <= 4'b0000 ;
    end
    else if (cnt_r==4'd9) begin      //计时10个cycle时，计时归0
      cnt_r        <=4'b0000;
    end
  else begin
      cnt_r        <= cnt_r + 1'b1 ; //计时加1
  end
end

  assign  cout = (cnt_r==4'd9) ;       //输出周期位
  assign  cnt  = cnt_r ;               //输出实时计时器

endmodule
```

看了这段代码，也许你云里雾里，或者之前没接触过硬件语言，心里有点打鼓。不过别担心，入门硬件语言并不难，我们按照代码顺序依次来看。

模块结构

首先，让我们看一看这段代码的第一行和最后一行。没错，一个模块的定义是以关键字module开始，以endmodule结束。module关键字后面跟的counter就是这个模块的名称。

看着有没有熟悉的感觉？你可能觉得，这个看着跟其他编程语言的函数定义也没多大区别吧？别急着下结论，再仔细看看接口部分，发现没有？这就和函数传入的参数很不一样了。

```
module counter(
//接口部分
input          reset_n,
input            clk,
output [3:0]     cnt,
output           cout
);
...... //逻辑功能部分
endmodule
```

Verilog模块的接口必须要指定它是输入信号还是输出信号。

输入信号用关键字input来声明，比如上面第4行代码的 input clk；输出信号用关键字output来声明，比如代码第5行的output [3:0] cnt；还有一种既可以输入，又可以输出的特殊端口信号，这种双向信号，我们用关键字inout来声明。

数据类型

前面我提到过，在可综合的Verilog代码里，基本所有写出来的东西都会对应实际的某个电路。而Verilog代码中定义的数据类型就能充分体现这一点。

```
parameter SIZE  =  2'b01;
reg [3:0]          cnt_r;
wire [1:0] cout_w;
```

比如上面代码的第9行，表示定义了位宽为4 bit 的寄存器reg类型信号，信号名称为cnt_r。

寄存器reg类型表示抽象数据存储单元，它对应的就是一种寄存器电路。reg默认初始值为X（不确定值），换句话说就是，reg电路在上电之后，输出高电平还是低电平是不确定的，一般是在系统复位信号有效时，给它赋一个确定值。比如例子中的cnt_r，在复位信号reset_n等于低电平时，就会给cnt_r赋“0”值。

reg类型只能在always和inital语句中被赋值，如果描述语句是时序逻辑，即always语句中带有时钟信号，寄存器变量对应为触发器电路。比如上述定义的cnt_r，就是在带clk时钟信号的always块中被赋值，所以它对应的是触发器电路；如果描述语句是组合逻辑，即always语句不带有时钟信号，寄存器变量对应为锁存器电路。

我们常说的电子电路，也叫电子线路，所以电路中的互连线是必不可少的。Verilog代码用线网wire类型表示结构实体（例如各种逻辑门）之间的物理连线。wire类型不能存储数值，它的值是由驱动它的元件所决定的。驱动线网类型变量的有逻辑门、连续赋值语句、assign等。如果没有驱动元件连接到线网上，线网就默认为高阻态“Z”。

为了提高代码的可读性和可维护性，Verilog还定义了一种参数类型，通过parameter来声明一个标识符，用来代表一个常量参数，我们称之为 **符号常量**，即标识符形式的常量。这个常量，实际上就是电路中一串由高低电平排列组成的一个固定数值。

数值表达

说到数值，我们再了解一下Verilog中的数值表达。还是以前面的4位十进制计数器代码为例，我们定位到第13行代码：

```
cnt_r       <= 4'b0000;
```

这行代码的意思是，给寄存器cnt_r赋以4’b0000的值。

这个值怎么来的呢？其中的逻辑“0”低电平，对应电路接地（GND）。同样的，逻辑“1”则表示高电平，对应电路接电源VCC。除此之外，还有特殊的“X”和“Z”值。逻辑“X”表示电平未知，输入端存在多种输入情况，可能是高电平，也可能是低电平；逻辑“Z”表示高阻态，外部没有激励信号，是一个悬空状态。

当然，为了代码的简洁明了，Verilog可以用不同的格式，表示同样的数值。比如要表示4位宽的数值“10”，二进制写法为4’b1010，十进制写法为4’d10，十六进制写法为4’ha。这里我需要特殊说明一下，数据在实际存储时还是用二进制，位宽表示储存时二进制占用宽度。

运算符

接下来我们看看Verilog的运算符，对于运算符，Verilog和大部分的编程语言的表示方法是一样的。

比如算术运算符 + - * / %，关系运算符 > < <= >= == !=，逻辑运算符 && || ! （与或非），还有条件运算符 ？ ，也就是C语言中的三目运算符。例如a?b:c，表示a为真时输出b，反之为c。

但在硬件语言里，位运算符可能和一些高级编程语言不一样。其中包括 ~ & | ^（按位取反、按位与，按位或，以及异或）；还有移位运算符，左移 << 和右移>> ，在生成实际电路时，左移会增加位宽，右移位宽保存不变。

条件、分支、循环语句

还有就是条件语句if和分支语句case，由于它们的写法和其它高级编程语言几乎一样，基本上你掌握了某个语言都能理解。

这里我们重点来对比不同之处，也就是用Verilog实现条件、分支语句有什么不同。用if设计的语句所对应电路是有优先级的，也就是多级串联的MUX电路。而case语句对应的电路是没有优先级的，是一个多输入的MUX电路。设计时，只要我们合理使用这两个语句，就可以优化电路时序或者节省硬件电路资源。

此外，还有循环语句，一共有 4 种类型，分别是 while，for，repeat和 forever 循环。注意，循环语句只能在 always 块或 initial 块中使用。

过程结构

下面我们来说说过程结构，最能体现数字电路中时序逻辑的就是always语句了。always 语句块从 0 时刻开始执行其中的行为语句；每当满足设定的always块触发条件时，便再次执行语句块中的语句，如此循环反复。

因为always 语句块的这个特点，芯片设计师通常把always块的触发条件，设置为时钟信号的上升沿或者下降沿。这样，每次接收到一个时钟信号，always块内的逻辑电路都会执行一次。

前面代码例子第11行的always语句，就是典型的时序电路设计方法，有没有感觉到很巧妙？

```
always@(posedge clk or negedge rstn)  begin

……    //逻辑语句

end
```

还有一种过程结构就是initial 语句。它从 0 时刻开始执行，且内部逻辑语句只按顺序执行一次，多个 initial 块之间是相互独立的。理论上，initial 语句是不可以综合成实际电路的，多用于初始化、信号检测等，也就是在编写验证代码时使用。

到这里，在我看来比较重要的Verilog基础知识就讲完了，这门语言的知识脉络我也为你搭起了骨架。当然了，Verilog相关知识远远不止这些。如果你对深入学习它很感兴趣，推荐你翻阅《Verilog HDL高级数字设计》等相关资料拓展学习。

总结回顾

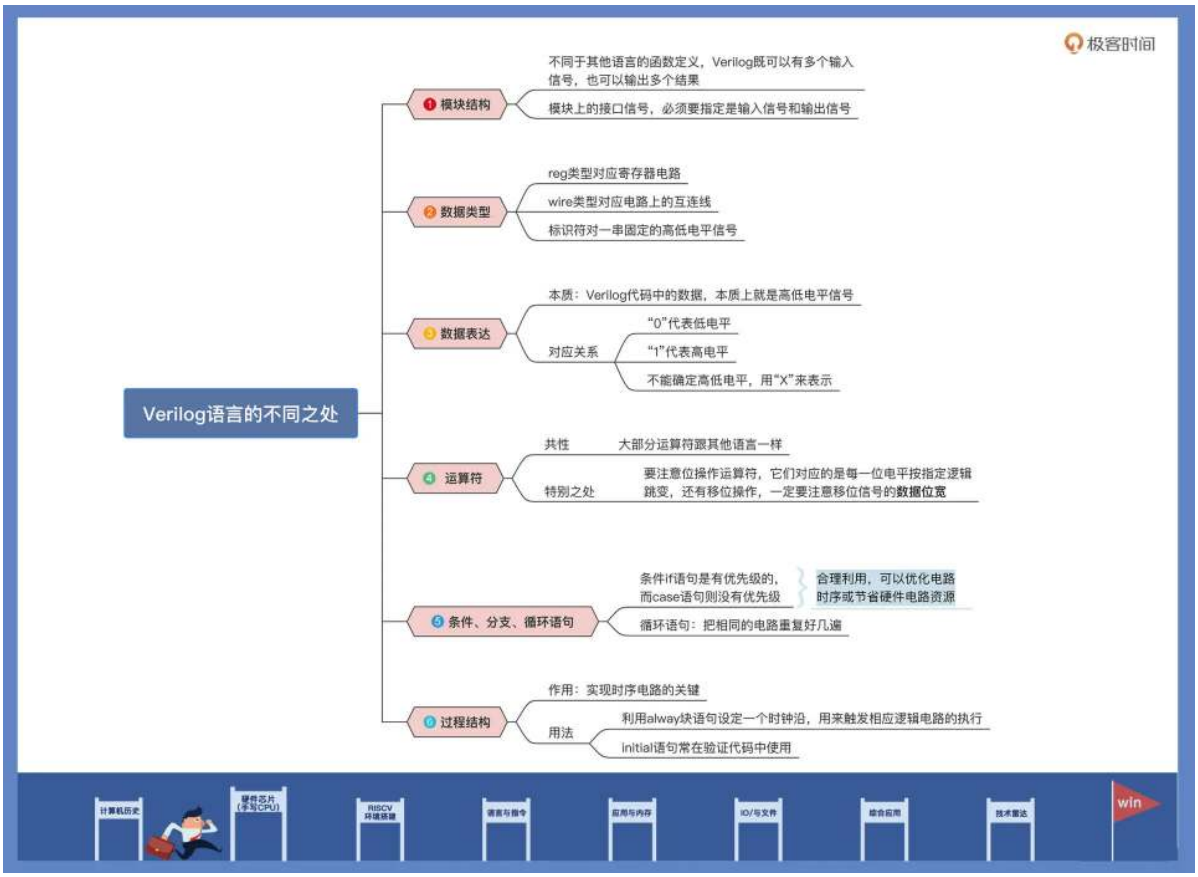
今天是芯片模块的第一节课，我们先了解了芯片的内部电路结构。一个芯片的内部电路往往分为数字电路模块和模拟电路模块。对于数字电路模块，可以使用Verilog硬件描述语句进行设计。

尽管Verilog这样的硬件语言你可能不大熟悉，但只要抓住本质，再结合代码例子建立知识脉络，学起来就能事半功倍。

要想熟悉硬件语言，我们最关键的就是做好思路转换。硬件语言跟高级编程语言本质的不同就是，使用Verilog的时候，必须理解每条语句实质上对应的什么电路，并且要从 **电路的角度** 来思考它为何要这样设计，而高级编程语言通常只要实现功能就行。

我再带你回顾一下，Verilog语言和高级编程语言具体有哪些不同：

1. **模块结构**：Verilog的模块结构和其他语言的函数定义不一样，它既可以有多个输入信号，也可以输出多个结果。而且，模块上的接口信号，必须要指定是输入信号和输出信号。
2. **数据类型**：跟我们在高级编程语言见到的变量类型相比，Verilog定义的数据类型也有很大不同。reg类型对应的是寄存器电路，wire类型对应的是电路上的互连线，标识符对应的是一串固定的高低电平信号。
3. **数据表达**：Verilog代码中的数据，本质上就是高低电平信号。“0”代表低电平，“1”代表高电平，不能确定高低电平的就用“X”来表示。
4. **运算符**：Verilog中的大部分运算符和其他语言是一样的，但是要注意位操作运算符，它们对应的是每一位电平按指定逻辑跳变，还有移位操作，一定要注意移位信号的数据位宽。
5. **条件、分支、循环语句**：Verilog中的条件if语句是有优先级的，而case语句则没有优先级，合理利用它们可以优化电路时序或节省硬件电路资源。循环语句则是把相同的电路重复好几遍。
6. **过程结构**：这是实现时序电路的关键。我们可以利用alway块语句设定一个时钟沿，用来触发相应逻辑电路的执行。这样，我们就可以依据时钟周期来分析电路中各个信号的逻辑跳变。而initial语句常在验证代码中使用，它可以从仿真的0时刻开始设置相关信号的值，并将这些值传输到待验证模块的端口上。



下节课，我会带你设计一个简单的电路模块，既能帮你复习今天学到的知识，还能通过实践体会一下代码是怎样生成电路的，敬请期待。

思考题

为什么很多特定算法，用Verilog设计并且硬件化之后，要比用软件实现的运算速度快很多？

欢迎你在留言区跟我交流互动，也推荐你把这节课分享给更多朋友。

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found

404 - Not found